

Marketplace Order & Fulfillment System

This document resolves all P0/P1 gaps identified in the audit (GAP-F1-F12, GAP-B1-B11) and defines the frontend system as it must exist at completion. It assumes the NestJS backend described in the audit (`/api` prefix, TypeORM entities, BullMQ queues) as the fixed integration surface, with **GAP-B1 (checkout endpoint mismatch) treated as the single hard blocker** that gates all buyer-flow work.

[UI/UX Design System & Theme Foundations](#)
[Information Architecture & Navigation](#)
[State Management & API Integration Strategy](#)
[Critical Remediation Implementation Specs](#)
[Component Library & File Tree Architecture](#)
[Verification & Definition of Done](#)

1.1 Design Philosophy

Both modes share: the same color tokens, the same status-badge vocabulary, the same 8px spacing scale, and the same type ramp — only density and layout containers differ. This is enforced by a single `theme.css` + Tailwind config consumed by both layout trees, so a `FULFILLED` badge looks and behaves identically whether it's rendered on a buyer's order page or an admin's dead-letter table.

Progressive density. Buyer screens never show a correlation ID, job ID, or retry count. Those only appear from `/vendor/orders` inward.

Root tokens (CSS variables), consumed via `tailwind.config.ts` `extend.colors` using `hsl(var(--token))`:



CSS

```
/* app/globals.css */
root {
  /* Surfaces */
  --background: 0 0% 100%;
  --surface: 220 20% 99%;
  --surface-raised: 0 0% 100%;
  --surface-sunken: 220 20% 97%;
  --border: 220 13% 91%;
  --border-strong: 220 13% 82%;

  /* Text */
  --foreground: 222 47% 11%;
  --foreground-muted: 215 16% 47%;
  --foreground-subtle: 215 16% 65%;

  /* Brand / primary action */
  --primary: 243 75% 59%; /* indigo-600 */
  --primary-foreground: 0 0% 100%;
  --primary-hover: 243 75% 52%;

  /* Contextual alerts */
  --info: 199 89% 48%;
  --info-bg: 199 89% 96%;
  --success: 142 71% 45%;
  --success-bg: 142 76% 96%;
  --warning: 38 92% 50%;
  --warning-bg: 48 96% 95%;
  --danger: 0 72% 51%;
  --danger-bg: 0 86% 97%;

  /* Focus / interaction */
  --ring: 243 75% 59%;
  --radius: 0.5rem;
}

dark {
  --background: 222 47% 7%;
  --surface: 222 42% 9%;
  --surface-raised: 222 40% 12%;
  --surface-sunken: 222 47% 6%;
  --border: 217 19% 20%;
  --border-strong: 217 19% 27%;
  --foreground: 210 20% 96%;
  --foreground-muted: 215 16% 65%;
  --foreground-subtle: 215 12% 45%;
  --info-bg: 199 60% 14%;
  --success-bg: 142 40% 12%;
  --warning-bg: 38 45% 13%;
  --danger-bg: 0 45% 14%;
}
```

Status Badge Vocabulary — this is the single canonical mapping every component (buyer, vendor, admin) imports from `lib/status-tokens.ts`. It unifies `Order.status`, `OrderLineItem.fulfillmentStatus`, and `VendorSyncJob.status` into one visual language so a user never has to learn three different color codes:

Status Key	Domain	Badge Color Token	Tailwind Class	Icon (lucide-react)
PLACED	Order	--info / --info-bg	bg-sky-50 text-sky-700 border-sky-200	Clock
CONFIRMED	Order	--info / --info-bg	bg-sky-50 text-sky-700 border-sky-200	CheckCircle2
FULFILLING / PROCESSING	Order, Line Item, Sync Job	--warning / --warning-bg	bg-amber-50 text-amber-800 border-amber-200	Loader2 (spin)
PENDING	Line Item, Sync Job	--foreground-subtle bg --surface-sunken	bg-slate-100 text-slate-600 border-slate-200	Hourglass
FULFILLED / COMPLETED	Order, Line Item	--success / --success-bg	bg-emerald-50 text-emerald-700 border-emerald-200	CheckCircle2
AMBIGUOUS / PENDING_RECONCILIATION	Sync Job	custom violet — 250 60% 55% / 250 60% 96%	bg-violet-50 text-violet-700 border-violet-200	HelpCircle
FAILED	Line Item, Sync Job	--danger / --danger-bg	bg-red-50 text-red-700 border-red-200	XCircle
DEAD_LETTER	Sync Job	--danger bg --surface-sunken, bold border	bg-red-100 text-red-800 border-red-300 border-2	AlertTriangle
MANUAL_INTERVENTION_REQUIRED	Order (derived)	--danger on --warning-bg (dual-tone, highest visual priority)	bg-amber-100 text-red-800 border-red-300	Siren
CANCELLED	Order	--foreground-subtle bg --surface-sunken	bg-slate-100 text-slate-500 border-slate-200 line-through-label	Ban

`MANUAL_INTERVENTION_REQUIRED` is a **derived, frontend-computed** state (not a raw backend enum value): an order is flagged this way in the UI whenever `order.lineItems.some(li => li.fulfillmentStatus === 'DEAD_LETTER' || li.fulfillmentStatus === 'AMBIGUOUS' with syncJob.attempts >= maxAttempts)`. It is the visual anchor of `/admin/orders?status=stuck`.

Badge component contract (`components/ui/status-badge.tsx`):

```

❏
tsx
type StatusBadgeProps = {
  status: OrderStatus | FulfillmentStatus | SyncJobStatus | 'MANUAL_INTERVENTION_REQUIRED';
  size?: 'sm' | 'md';
  pulse?: boolean; // true for PROCESSING/FULFILLING — animate-pulse on the dot, not the whole badge
};

```

Rule: only the leading status-dot animates (`animate-pulse`), never the badge background or text — prevents dashboard flicker fatigue at high density.

1.3 Scaffolding & Component Hierarchy

Layout containers:

`<AppShell>` — top-level: header, optional sidebar slot, `<main>` with `max-w-7xl mx-auto px-4` (storefront) or `w-full` fluid (operational).
`<StorefrontHeader>` — logo, nav (`Products`), cart icon with item-count badge, sticky top, `h-16`, `border-b border-border bg-surface/80 backdrop-blur`.

`<OperationalSidebar>` — collapsible (`(w-64 / w-16)`), used by both `/vendor/*` and `/admin/*`, driven by a `role` prop that swaps the nav item list only.

`<PageHeader>` — title, description, right-aligned action slot (buttons), used on every list/detail screen for consistency.

`<DataTable>` — generic, used by `/admin/orders`, `/admin/dead-letter`, `/admin/audit-logs`, `/vendor/orders`, `/vendor/dead-letter`. Column defs, sticky header, row click-through, built-in empty/loading/error slots (see below).

Loading skeletons (`(components/ui/skeleton.tsx)`, primitive `<Skeleton className="animate-pulse bg-surface-sunken rounded-md" />`):

`<CatalogGridSkeleton />` — 8 shimmering product cards, matches real card aspect ratio exactly (prevents layout shift).

`<OrderDetailSkeleton />` — header block + 2-4 line-item row placeholders.

`<TableRowSkeleton rows={n} columns={n} />` — generic, used by every `<DataTable>` default loading state.

Toast notifications (`(components/ui/toast.tsx)`, built on Radix Toast, positioned bottom-right on desktop / top on mobile):

Variants: `success`, `error`, `info`, `warning` — mapped 1:1 to the contextual-alert tokens above.

Actionable toasts: accept an optional `{ label, onClick }` — e.g. inventory update failure toast includes a "Retry" action; dead-letter retry-triggered toast includes a "View Job" action that deep-links to `/admin/dead-letter?jobId=...`.

Toasts never auto-dismiss on `error` or `warning` variant (closes part of GAP-F9's "dismissal requires manual action" inconsistency — the fix is to make this the *documented, intentional* behavior for destructive/failure states, while `success`/`info` auto-dismiss at 4s).

Empty states (`(components/ui/empty-state.tsx)`):

```
<EmptyState
  icon={PackageOpen}
  title="No orders yet"
  description="Orders you place will show up here."
  action={{ label: 'Browse products', href: '/products' }}
/>
```

Every `<DataTable>` and grid consumer must supply an `EmptyState` config; there is no default so absence is a compile-time-visible gap during review.

Error boundaries (closes GAP-F8): Next.js App Router `error.tsx` at every route segment listed in §2, each rendering

```
<RouteErrorFallback
  error={error}
  reset={reset}
  title="Something went wrong loading this page"
  supportHint="If this keeps happening, note the time and refresh."
/>
```

`RouteErrorFallback` logs `error.digest` to the console in dev and to the analytics sink in prod, shows a "Try again" button wired to `reset()`, and a secondary "Go to homepage" link — it never shows a raw stack trace to end users, only in a collapsible `<details>` in dev mode (`(NODE_ENV !== 'production')`).

2. Information Architecture & Navigation

2.1 Route Map



→ redirect → /products [fixes GAP-F11]	
BUYER STOREFRONT	
/products	Catalog Grid
products/[id]	Product Detail Page [fixes GAP-F3]
checkout	Checkout Form
orders/[id]	Order Confirmation + Live Tracking [fixes GAP-F1]
VENDOR PORTAL (role-gated: vendorId from session/context)	
vendor/inventory	Stock Management
vendor/orders	Vendor Order Queue (polling)
vendor/dead-letter	Vendor-scoped DLQ [fixes GAP-F12]
ADMIN CONSOLE (role-gated: admin)	
admin	→ redirect → /admin/dashboard
admin/dashboard	System Health Overview
admin/orders	Global Order Management
admin/orders/[id]	Order Detail + Audit Timeline
admin/dead-letter	Global DLQ Management
admin/audit-logs	Correlation-ID Trace Explorer

2.2 Screen Specifications

/products — Catalog Grid

Layout: `<AppShell><StorefrontHeader>` + responsive grid (`grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6`).

Card contents: product image (placeholder gradient if none), name, vendor name (muted, links nowhere — informational), price,

Stock Badge: `In Stock` (`--success-bg`) if `stockCount > 10`, `Low Stock (n left)` (`--warning-bg`) if `1 ≤ stockCount ≤ 10`, `Out of Stock` (`--danger-bg`, card dimmed to 60% opacity, CTA disabled) if `0`.

Filters: sticky filter bar above grid — vendor multi-select, price range slider, "In stock only" toggle. Filter state lives in URL search params (`?vendor=...&inStock=true`) so filtered views are shareable/bookmarkable.

CTA: card click → `/products/[id]`. "Add to Cart" quick-action button on hover (desktop) / always visible (mobile) adds default qty 1 without navigating.

API: `GET /api/catalog` (with query params mapped from filter state), `React Query` key `['catalog', filters]`.

/products/[id] — Product Detail Page (GAP-F3 fix)

Layout: two-column desktop (`image | details`), stacked mobile.

Details column, top to bottom: vendor name, product name (`text-2xl font-semibold`), price, **Inventory Lock Indicator** (see §4.2), quantity stepper (clamped `[1, min(stockCount, 20)]`), "Add to Cart" primary CTA, "Buy Now" secondary CTA (adds to cart + navigates to `/checkout`).

Below fold: description, vendor's other products (rail).

Edge case: `stockCount === 0` → CTA replaced with disabled `Out of Stock` button + "Notify me" ghost button (non-functional placeholder acceptable for v1, but must render, not disappear).

API: `GET /api/catalog/:id`. On 404 → renders Next.js `not-found.tsx` (distinct from generic error boundary).

/checkout — Checkout Form (GAP-F2, GAP-F5, GAP-B1 fix — see §4.1)

Layout: single column, `max-w-2xl mx-auto`, two sections: **Shipping** (fieldset) and **Order Summary** (sticky sidecar on desktop, collapsible accordion on mobile).

Fields: shipping address (multi-line: street, city, state/region, postal code, country — combined client-side into the single `shippingAddress` string the DTO expects), buyer identifier (pre-filled from session, read-only).

Live Total: recalculates as cart quantities change; itemized by vendor (multi-vendor cart shows vendor sub-groups, matching FR4's multi-vendor line-item model).

Submit button states: `Idle` → "Place Order", `Submitting` → "Placing your order..." (spinner, disabled), `Error` → "Try again" (re-enabled, error banner above button).

Non-interactive during submit: entire `<fieldset disabled={isSubmitting}>` wraps the form — prevents double-submit and mid-flight field edits (closes GAP-F5's "no confirmation/validation" gap for the submit-race case specifically; general Zod validation covered in

§3.2).

`/orders/{id}` — Order Confirmation + Live Tracking (*GAP-F1, GAP-F6 fix — see §4.1*)

Layout: header card (order ID, placed date, overall `Order Status` badge, "Track Order" anchor if arrived via email/deep-link) + line-item list grouped by vendor, each row showing product name, qty, unit price, and its own `Fulfillment Status` badge.

Polling: see §3.2 for the hook contract. A subtle `<PollingIndicator>` (small dot, `animate-pulse` in `--info`) sits next to the page title while polling is active; disappears once status reaches a terminal state, replaced by a static "Last updated" timestamp.

Ambiguous state UX: if any line item is `AMBIGUOUS`, render an inline `<Alert variant="info">` banner: "We're confirming this item with the vendor — this can take a few minutes." — never phrased as an error, per PRD §7.5 (do not assume timeout = failure).

Dead-letter state UX: if any line item is `DEAD_LETTER`, render `<Alert variant="danger">`: "There was a problem fulfilling this item. Our team has been notified." — no technical detail (job IDs, retry counts) surfaced to buyers.

`/vendor/inventory` — Stock Management

Layout: `<DataTable>` — columns: Product, Price, Stock Count (inline-editable number input), Version (hidden by default, visible via "Show technical columns" toggle for debugging optimistic-lock conflicts), Active toggle, Last Updated.

Optimistic lock feedback: inline edit → `PATCH /api/catalog/:id` sends current `version`. On `409 Conflict` (version mismatch), the row flashes `--warning-bg`, input reverts to server value, and a toast reads: "This item was updated elsewhere — showing the latest value." Row auto-refetches.

`/vendor/orders` — Vendor Order Queue (*GAP-F6 fix — see §4.1*)

Layout: `<DataTable>` filtered to the current vendor's line items across all orders. Columns: Order ID (short, `#a1b2c3`), Buyer (masked/short ID — no PII beyond what buyer flow exposes), Product, Qty, Fulfillment Status badge, Last Sync Attempt, Actions.

Polling with pause/resume: header-right toggle switch `Live updates: On/Off`, defaults On, 5s interval; explicit "Refresh" icon-button remains for manual pull regardless of toggle state.

Terminal state detection: rows whose status is `FULFILLED`, `FAILED`, `DEAD_LETTER`, or `CANCELLED` stop being included in the polled diff-set (visually: they simply stop flickering/re-rendering) — this is a row-level optimization layered on top of the page-level polling described in §4.1.

`/vendor/dead-letter` — Vendor-scoped DLQ (*GAP-F12 fix*)

Layout: `<DataTable>` scoped to `vendorId`, columns: Order Line Item, Product, Attempts / Max Attempts, Last Error Message (truncated, expandable), Failed At.

No retry action for vendors — retry is an admin-only privileged action (`POST /admin/dead-letter/:jobId/retry`); vendors get read-only visibility plus a "Contact support" mailto/CTA. This is a deliberate authorization boundary, not an oversight.

API: `GET /api/fulfillment/dead-letter?vendorId=...`.

`/admin/dashboard` — System Health Overview

Layout: KPI card row (4-up grid): `Orders (24h)`, `Dead-Letter Count`, `Ambiguous / Pending Reconciliation`, `Mean Time to Reconciliation`. Below: two panels — "Orders needing attention" (top 5 `MANUAL_INTERVENTION_REQUIRED` orders, link to `/admin/orders?status=stuck`) and "Queue throughput" mini sparkline chart (from `/api/fulfillment/queues/stats`).

API: `GET /api/admin/dashboard`.

`/admin/orders` — Global Order Management (*GAP-F4 fix — see §4.3*)

Layout: `<DataTable>` with filter bar: status dropdown (includes synthetic `stuck` filter mapping to `MANUAL_INTERVENTION_REQUIRED`), buyer ID search, date range. Columns: Order ID, Buyer, Vendor(s) (badge list if multi-vendor), Status, Line Items (count), Placed At, Actions (`View`, `Cancel`).

Line-item override: row-expand (`chevron` click) reveals nested mini-table of line items with an inline "Resolve" action per stuck line item, opening `<ResolveLineItemDialog>` (radio: `Mark Fulfilled` / `Mark Failed` / `Retry Sync`, required `reason` textarea) → `POST /admin/line-items/:lineItemId/resolve`.

`/admin/orders/{id}` — Order Detail + Audit Timeline

Layout: order summary card (same visual component as buyer's `/orders/{id}` header, reused, plus admin-only correlation ID chip that copies to clipboard on click) + line items table (with resolve actions inline) + **Audit Log Timeline** — vertical timeline component, each entry: action label, actor, timestamp, expandable JSON diff (`previousState` → `newState`) rendered as a two-column diff, not raw JSON dump.

API: `GET /api/orders/:id`, `GET /api/orders/:id/audit-logs`.

Layout: `<DataTable>`, filter by vendor + date range. Columns: Job ID, Order Line Item, Vendor, Attempts, Error Message, Failed At, **Retry** button.

One-click retry: button → optimistic row update to `PENDING` (grey, spinner) → `POST /admin/dead-letter/:jobId/retry` → on success, toast + row updates to reflect new job status on next poll tick; on failure, row reverts + error toast with retry-the-retry action.

Layout: search bar (correlation ID input, primary), results render as the same `<AuditTimeline>` component used on `/admin/orders/[id]`, plus a summary header showing the full trace span: `checkout → inventory decrement → vendor sync → fulfillment confirmation` as a horizontal stepper with timestamps at each stage (directly visualizes the PRD's Observability NFR).

API: `GET /admin/trace/:correlationId` (preferred, full trace) with `GET /admin/audit-logs` as the fallback list/browse view when no correlation ID is provided yet.

2.3 Navigation & Role Gating

Storefront and operational shells are separate route groups: `app/(storefront)/...` and `app/(operational)/vendor/...`, `app/(operational)/admin/...`, sharing `<OperationalSidebar>` with a `role` prop. `middleware.ts` gates `/vendor/*` and `/admin/*` by session role claim; unauthorized access redirects to `/products` with a toast, not a raw 403 page — keeps the failure mode consistent with the “no silent failure, no scary raw errors” principle in §1.1.

3. State Management & API Integration Strategy

3.1 Client-Side State — Zustand

`context/CartStore.ts` (closes GAP-F7, GAP-F10):

ts

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';

interface CartLineItem {
  productId: string;
  vendorId: string;
  name: string;
  unitPrice: number;
  quantity: number;
  stockCountAtAdd: number; // snapshot for client-side clamp; server is source of truth at checkout
}

interface CartState {
  items: CartLineItem[];
  hydrated: boolean;
  addItem: (item: CartLineItem) => void;
  updateQuantity: (productId: string, quantity: number) => void;
  removeItem: (productId: string) => void;
  clear: () => void;
  setHydrated: (v: boolean) => void;
}

export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      items: [],
      hydrated: false,
      addItem: (item) => {
        set((state) => {
          const existing = state.items.find((i) => i.productId === item.productId);
          if (existing) {
            return {
              items: state.items.map((i) =>
                i.productId === item.productId
                  ? { ...i, quantity: Math.min(i.quantity + item.quantity, i.stockCountAtAdd) }
                  : i
              ),
            };
          }
          return { items: [...state.items, item] };
        });
      },
      updateQuantity: (productId, quantity) => {
        set((state) => ({
          items: state.items
            .map((i) => (i.productId === productId ? { ...i, quantity } : i))
            .filter((i) => i.quantity > 0),
        })),
      removeItem: (productId) => {
        set((state) => ({ items: state.items.filter((i) => i.productId !== productId) })),
      },
      clear: () => set({ items: [] }),
      setHydrated: (v) => set({ hydrated: v }),
    }),
    {
      name: 'marketplace-cart',
      storage: createJSONStorage(() => localStorage),
    }
  )
);
```

```
partialize: (state) => ({ items: state.items }), // never persist `hydrated`
onRehydrateStorage: () => (state) => state?.setHydrated(true),
}
)
};
```

SSR hydration safety: every component reading `useCartStore` for render-affecting values (cart icon count, checkout summary) must gate on `hydrated`:

```

tsx
const { items, hydrated } = useCartStore();
if (!hydrated) return <CartIconSkeleton />; // never render `items.length` before hydration — avoids SSR/CSR mismatch flash of "0"
```

This is a hard rule, not a suggestion — the `hydrated` flag exists specifically to prevent the classic Zustand-persist SSR flash-of-empty-cart bug.

3.2 Data Fetching & Polling Protocol — TanStack Query

Base client setup (`lib/query-client.ts`):

```

ts
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: (failureCount, error) => {
        if (error instanceof ApiError && error.status >= 400 && error.status < 500) return false; // never retry 4xx
        return failureCount < 3;
      },
      retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 10000), // exponential backoff, capped 10s
      staleTime: 30_000,
    },
  },
});
```

Smart polling hook (`lib/hooks/use-order-polling.ts`) — the canonical implementation shared by `/orders/[id]` and `/vendor/orders`:

```

ts
```

```

const TERMINAL_ORDER_STATUSES = ['FULFILLED', 'CANCELLED'] as const;
const TERMINAL_LINE_ITEM_STATUSES = ['FULFILLED', 'FAILED', 'DEAD_LETTER', 'CANCELLED'] as const;

function isTerminal(order: OrderResponseDto): boolean {
  if (TERMINAL_ORDER_STATUSES.includes(order.status)) return true;
  return order.lineItems.every((li) =>
    TERMINAL_LINE_ITEM_STATUSES.includes(li.fulfillmentStatus)
  );
}

export function useOrderPolling(orderId: string, options?: { enabled?: boolean }) {
  const [isPollingActive, setIsPollingActive] = useState(true);

  const query = useQuery({
    queryKey: ['order', orderId],
    queryFn: () => api.get<OrderResponseDto>(`/orders/${orderId}`),
    enabled: (options?.enabled ?? true),
    refetchInterval: (query) => {
      if (!isPollingActive) return false;
      const data = query.state.data;
      if (data && isTerminal(data)) return false; // stop polling on terminal state
      return 5000;
    },
    refetchIntervalInBackground: false, // pause when tab is backgrounded — avoids wasted calls
  });

  return { ...query, isPolling: isPollingActive && !(query.data && isTerminal(query.data)), setIsPollingActive };
}

```

This single hook backs the buyer's `/orders/[id]` (always-on) and the vendor's `/vendor/orders` (`setIsPollingActive` wired to the pause/resume toggle from §2.2, closing GAP-F6).

Race condition prevention during checkout flight:

Submit button + fieldset disabled via `mutation.isPending` (TanStack `useMutation`) — prevents duplicate submits at the UI layer. `useMutation` is invoked with no automatic retry (`retry: false`) for the checkout mutation specifically — a failed checkout must never silently double-charge inventory via an auto-retry; the user explicitly re-triggers.

On success, `queryClient.setQueryData(['order', response.id], response)` seeds the order-detail cache immediately (before navigation), so `/orders/[id]` renders instantly with data already in cache instead of showing a loading skeleton for a page that just got its data.

Cart is cleared (`useCartStore.getState().clear()`) only after a **confirmed 2xx response**, never optimistically before the request resolves — an interrupted request (page refresh mid-flight) must leave the cart intact rather than losing the buyer's items.

3.3 API Client Contract Alignment

`lib/api.ts` is the single fetch wrapper; every call is typed against `lib/types.ts`, which mirrors the backend DTOs 1:1. This section is the explicit map that resolves GAP-B1/GAP-F2.

The critical row above is bolded because it is **GAP-B1/GAP-F2**: the frontend must call `checkoutOrder()` → `POST /api/orders/checkout` with a `CheckoutDto` body, never `api.post('/orders', order)`. `CancelOrderDto` (currently missing per GAP-B10) must be added on the backend alongside this frontend contract — the frontend implementation assumes it exists and will fail typecheck against the real backend until it does, which is intentional: it turns GAP-B10 into a build-time-visible blocker instead of a silent runtime gap.

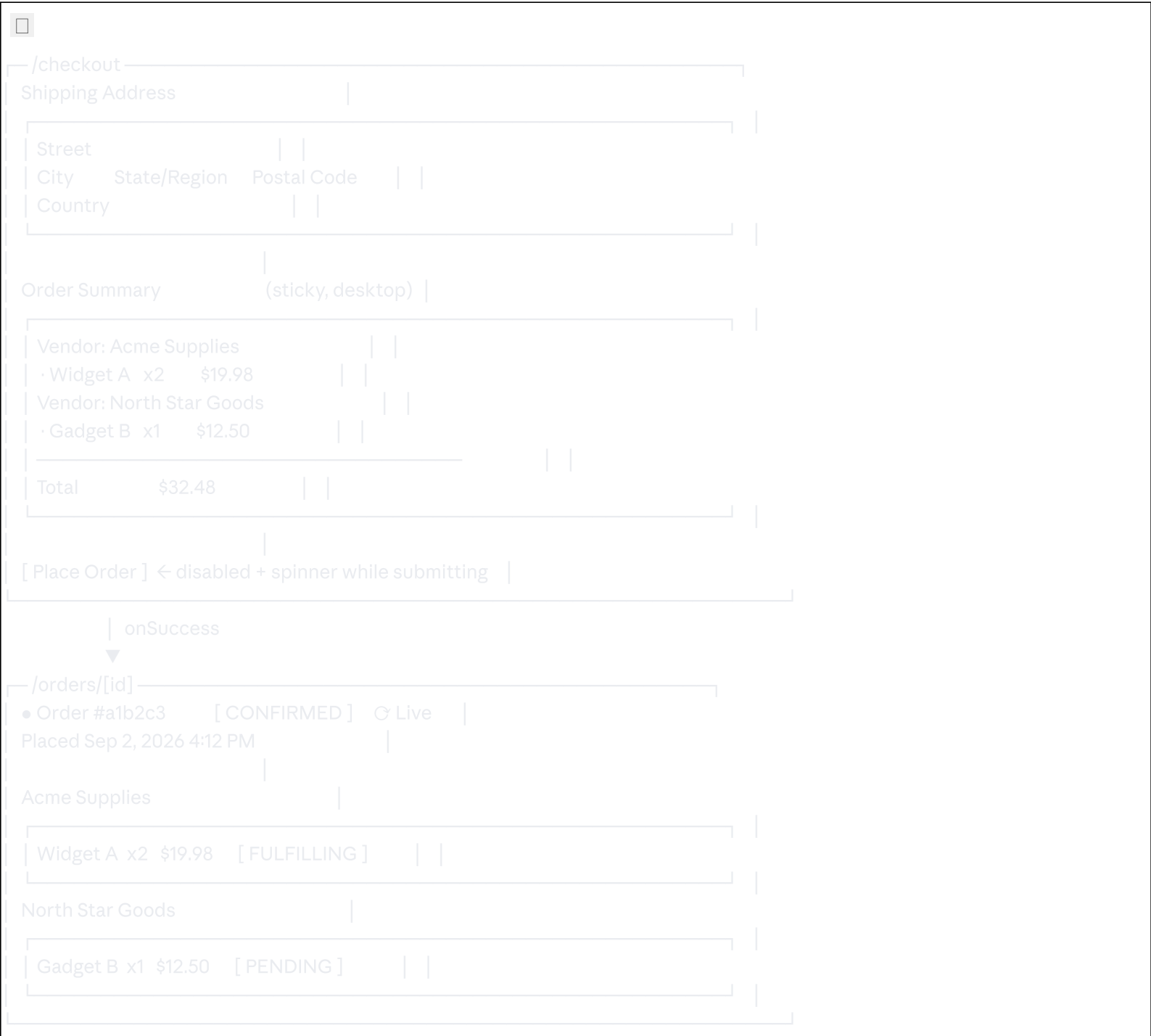
4. Critical Remediation Implementation Specs (P0 & P1 Targets)

4.1 GAP-F1 & GAP-B1 — Checkout → Order Confirmation Flow

Root cause (per audit): frontend calls `POST /orders`; backend only exposes `POST /orders/checkout`. This is a pure contract mismatch,

not a missing feature — the fix is entirely in `frontend/lib/api.ts` and `frontend/app/checkout/page.tsx`; `CheckoutDto`/`CheckoutResponseDto` already exist server-side.

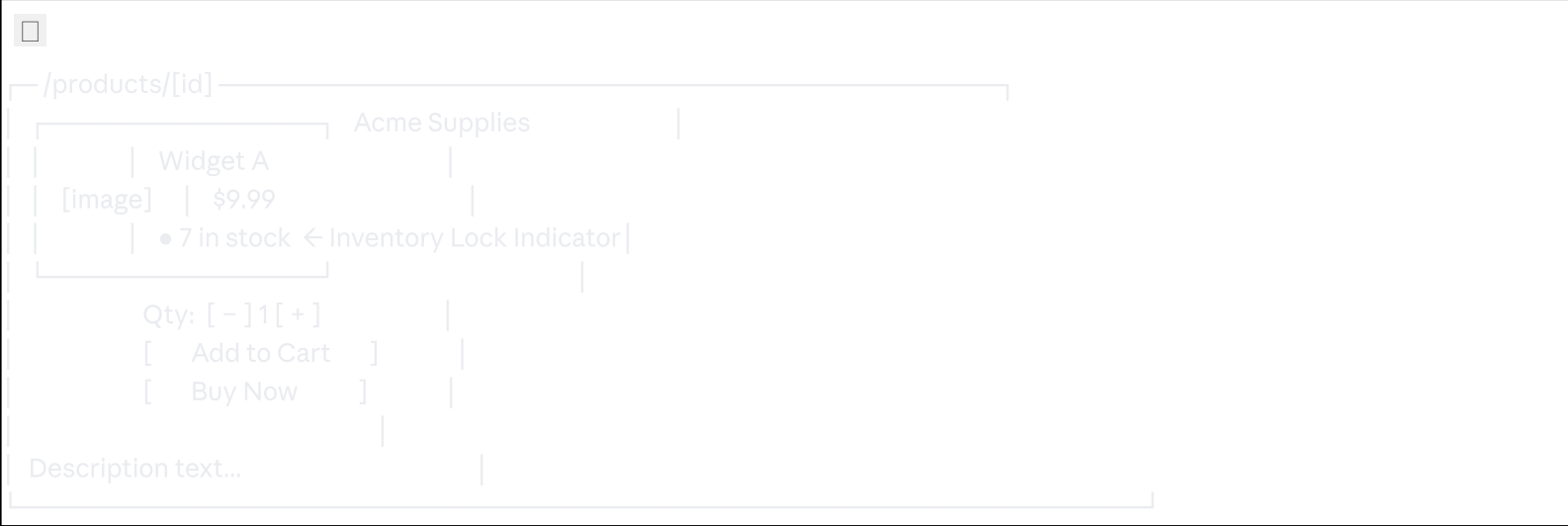
Flow wireframe:



```
const checkoutMutation = useMutation({
  mutationFn: (dto: CheckoutDto) => api.checkoutOrder(dto),
  retry: false,
  onSuccess: (order) => {
    queryClient.setQueryData(['order', order.id], order);
    useCartStore.getState().clear();
    router.push(`/orders/${order.id}`);
  },
  onError: (err) => {
    toast.error(mapCheckoutErrorToMessage(err)); // e.g. 409 stock conflict → "One of your items just sold out"
  },
});
```

`mapCheckoutErrorToMessage` special-cases the FR3 oversell-race outcome: a `409` from the backend's row-locked inventory check maps to a specific, actionable message rather than a generic "Something went wrong" — this is the user-facing edge of the concurrency guarantee described in the PRD.

4.2 GAP-F3 — Product Detail Page & Inventory Lock Indicator



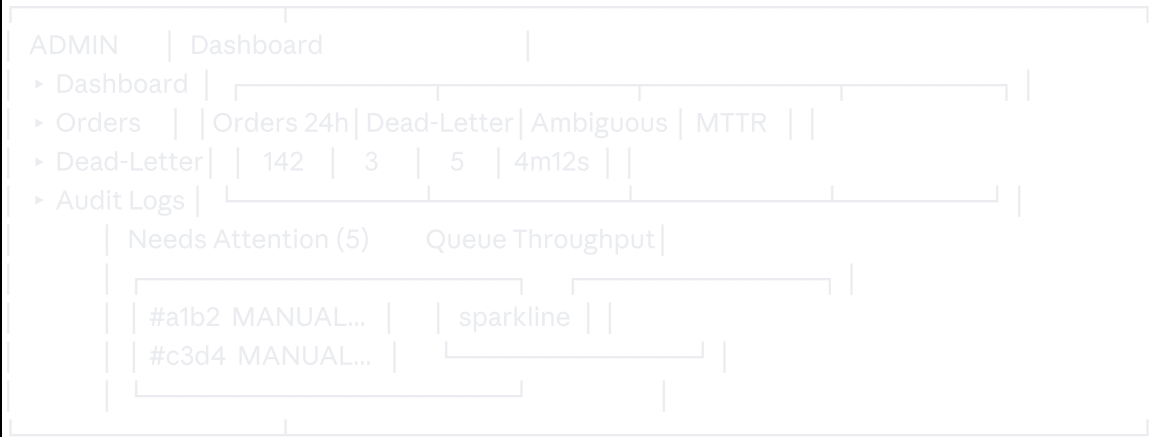
Inventory Lock Indicator is not literal UI for the `SELECT ... FOR UPDATE` row lock (that's a backend implementation detail buyers never see) — it is the *stock-count-as-of-last-fetch* badge, explicitly labeled as a soft guarantee:

- `≥ 11`: `● In stock` (`--success`)
- `1-10`: `● Only {n} left` (`--warning`)
- `0`: `● Out of stock` (`--danger`), CTA disabled

The quantity stepper's max is clamped client-side to the last-known `stockCount`, but the page copy under the stepper reads: "Availability is confirmed at checkout." — setting the correct expectation that the authoritative check (and possible 409) happens server-side per FR3, not on this page.

4.3 GAP-F4 — Admin Layout & Sub-System Screens



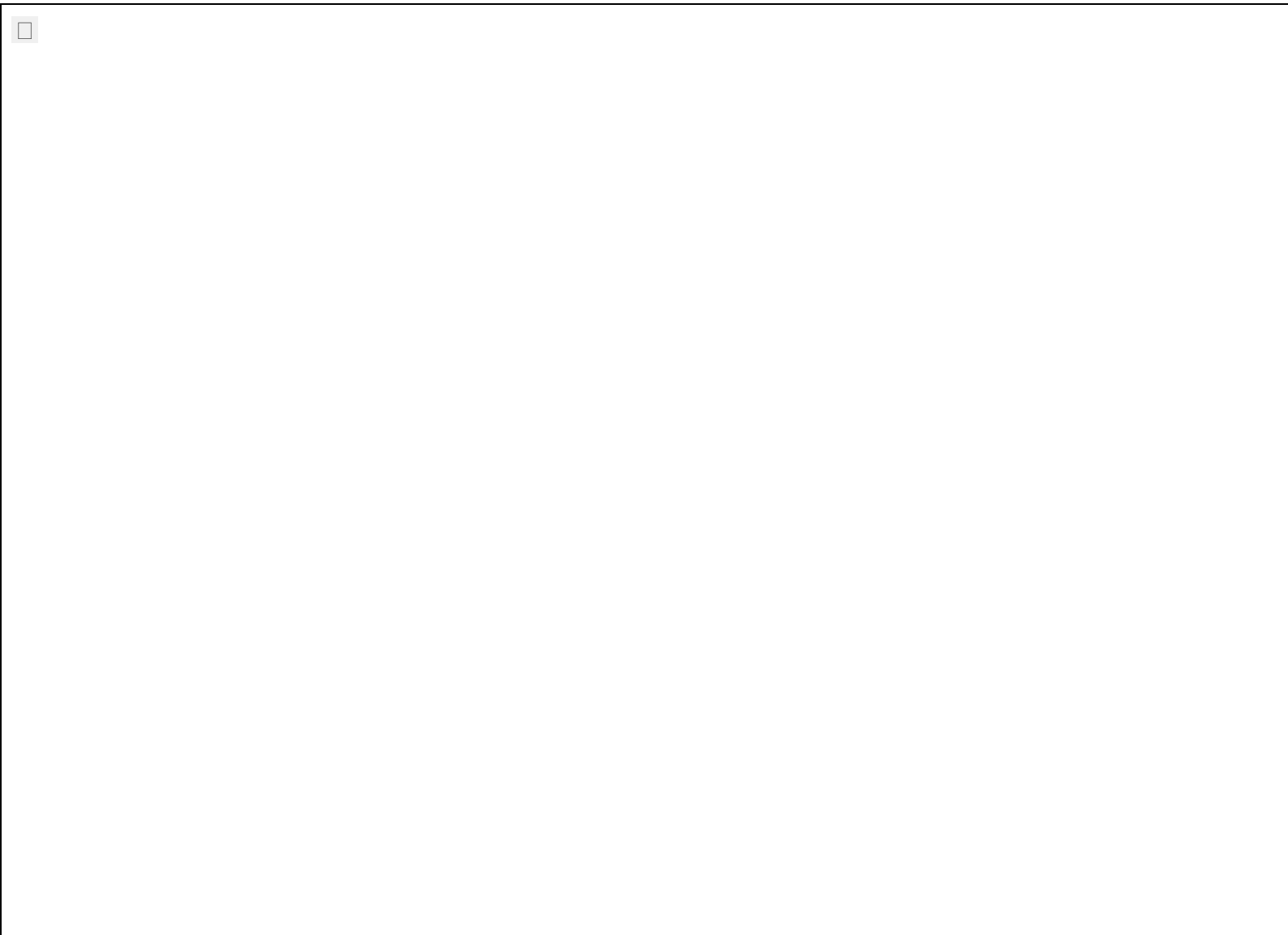


`app/(operational)/admin/layout.tsx` composes `<OperationalSidebar role="admin">` + `<AppShell fluid>`, wraps every child route in `<RequireRole role="admin">`, and provides an `error.tsx` sibling at the layout root so a crash in any admin sub-page falls back to `<RouteErrorFallback>` without taking down the sidebar/shell.

4.4 GAP-F8 — Error Boundaries per Route Segment

Applied at every level in §5's file tree where an `error.tsx` appears — the rule: **any segment that fetches data on its own gets its own `error.tsx`**, so a crash in `/admin/orders/[id]` doesn't blank out `/admin/orders`'s list, and a crash in one `<DataTable>` row-expand doesn't take out the whole admin shell. Root-level `app/error.tsx` remains as the final catch-all for anything above the segment boundaries (e.g., a `layout.tsx` crash itself).

5. Component Library & File Tree Architecture



6. Verification & Definition of Done (DoD)

6.1 TypeScript Strict Mode Requirements

`tsconfig.json` must have `"strict": true`, `"noUncheckedIndexedAccess": true`, `"exactOptionalPropertyTypes": true`.

CI gate: `npx tsc --noEmit` must exit `0` before merge. No `// @ts-ignore` in `app/` or `components/` without an inline justification comment referencing a tracked issue.

Every API client function in `lib/api.ts` must have an explicit, non-`any` return type sourced from `lib/types.ts`; `types.ts` fields must match backend DTOs field-for-field (this is what makes GAP-B10, the missing `CancelOrderDto`, surface as a compile error rather than a runtime 500).

`zustand` store selectors must be typed (no implicit `any` from `useCartStore((s) => s.items)` without the store's generic resolving correctly).

6.2 UX Resilience Checklist

Scenario	Required Behavior
Network timeout on <code>GET /api/catalog</code>	<code><CatalogGridSkeleton></code> → after retry exhaustion, <code><EmptyState></code> -style error card with "Retry" button, not a blank grid
<code>POST /api/orders/checkout</code> returns 409 (oversell race, FR3)	Specific toast: "One of your items just sold out" + cart line item flagged, not a generic error
<code>POST /api/orders/checkout</code> returns 5xx	Fieldset re-enabled, error banner above submit button, cart preserved, no navigation
Backend returns malformed/partial order (missing <code>lineItems</code>)	Defensive render: <code>order.lineItems ?? []</code> , empty-state message "No items found for this order" rather than a crash — caught at the type-guard boundary in <code>lib/api.ts</code> , not deep in JSX
Cold start — first request to a sleeping/scaling backend takes >3s	Loading skeleton persists past the normal timeout threshold with no flash of an error state before the actual timeout limit is hit; only escalate to error UI once retries are exhausted
Sandbox/demo environment with seeded-but-stale data	All timestamps rendered via relative-time (<code>formatDistanceToNow</code>) with an absolute-time tooltip, so stale seed data ("3 months ago") still reads coherently instead of looking broken
Order reaches <code>AMBIGUOUS</code> and stays there past a reasonable window	No special buyer-facing timeout UI is invented client-side — the buyer view continues showing the calm "confirming with vendor" message; only <code>/admin/dashboard</code> and <code>/admin/orders?status=stuck</code> escalate this, per the role-appropriate density principle in §1.1
Tab backgrounded during active polling	<code>refetchIntervalInBackground: false</code> — polling pauses; on refocus, TanStack Query's default <code>refetchOnWindowFocus</code> immediately catches up state
Zustand cart persisted from a previous session references now-deleted/inactive products	On cart-drawer open, cross-check persisted <code>productId</code> s against a lightweight <code>GET /api/catalog</code> batch lookup; stale items render with a "No longer available" badge and are excluded from the checkout total, not silently included

6.3 Definition of Done — per screen

A screen is "done" only when all of the following hold:

Loading, error, and empty states are implemented and visually distinct (§1.3).

Its route segment has a sibling `error.tsx` (§4.4).

All API calls go through `lib/api.ts` with types from `lib/types.ts` — no inline `fetch()`.

`npx tsc --noEmit` passes with the screen's files included.

Status badges, where applicable, use `components/ui/status-badge.tsx` exclusively — no ad-hoc color classes on status text anywhere in the codebase.

Polling screens (`/orders/[id]`, `/vendor/orders`) correctly stop polling on terminal state, verified manually against a seeded order driven through `FULFILLED`/`FAILED`/`DEAD_LETTER` via the admin resolve action.

Role-gated screens (`/vendor/*`, `/admin/*`) redirect (not 403-page) when accessed without the correct role.